

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software
[1INF54-0982]

Estándar de Diseño de APIs

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión de los estándares de programación web frontend.	Equipo 6F

Contenido

1. Introducción.....	4
2. Estilo arquitectónico.....	4
3. Convenciones de endpoints.....	4
3.1. Filtros, Búsquedas y Ordenamiento (Query Parameters).....	5
4. Códigos de estado HTTP.....	5
5. Estructura estándar de los JSON de respuesta.....	6
5.1. Respuesta exitosa.....	6
5.2. Respuesta de error.....	6
5.3. Convenciones de nombres de campos.....	7
6. Contratos y DTOs.....	7
7. Cabeceras y Seguridad.....	7
8. Webhooks y Eventos Externos.....	7
9. Referencias y recursos.....	7

1. Introducción

Este documento define las reglas de diseño de la API que comunica el backend del planificador (Java/Spring Boot) con el frontend del visualizador (React/TypeScript) del sistema SysMile. El objetivo es garantizar consistencia, predictibilidad y facilidad de consumo de los endpoints por parte del equipo de frontend.

2. Estilo arquitectónico

- La comunicación entre frontend y backend se realiza mediante una API REST sobre HTTP/JSON.
- Se utiliza WebSocket (STOMP sobre SockJS) exclusivamente para la actualización en tiempo real de la posición de los vehículos y el estado de las rutas en el mapa; toda operación de creación, consulta, actualización o eliminación de recursos se realiza mediante REST.
- Todas las rutas de la API se versionan con el prefijo `/api/v1/`, permitiendo introducir `/api/v2/` en el futuro sin romper a los consumidores existentes.

3. Convenciones de endpoints

- Los recursos se nombran en plural y en kebab-case: `/api/v1/delivery-routes`, `/api/v1/packages`, `/api/v1/drivers`.
- Los recursos anidados reflejan la relación de pertenencia: `/api/v1/delivery-routes/{routeId}/stops`.
- No se incluyen verbos en la URL; la acción se expresa mediante el método HTTP correspondiente.

Método HTTP	Uso	Ejemplo
GET	Obtener uno o varios recursos, sin efectos secundarios	GET <code>/api/v1/delivery-routes/{id}</code>
POST	Crear un nuevo recurso, o disparar una acción de negocio (ej. optimizar)	POST <code>/api/v1/delivery-routes/{id}/optimize</code>
PUT	Reemplazar completamente un recurso existente	PUT <code>/api/v1/drivers/{id}</code>
PATCH	Actualizar parcialmente un recurso	PATCH <code>/api/v1/delivery-routes/{id}/status</code>

DELETE	Eliminar (o desactivar lógicamente) un recurso	DELETE /api/v1/packages/{id}
--------	--	---------------------------------

- Las acciones que no corresponden a una operación CRUD directa (ej. disparar el algoritmo ALNS/IPSO sobre una ruta) se modelan como un sub-recurso verbal bajo POST: POST /api/v1/delivery-routes/{id}/optimize.

3.1. Filtros, Búsquedas y Ordenamiento (Query Parameters)

- Los parámetros de búsqueda y filtrado se envían en la Query String usando camelCase (ej. /api/v1/packages?status=DELIVERED&driverId=uuid).
- Para el ordenamiento se utiliza el parámetro sort con el formato campo,dir (ej. sort=createdAt,desc).
- La paginación basada en cursor utiliza los parámetros limit y cursor.

4. Códigos de estado HTTP

Código	Significado	Cuándo usarlo
200 OK	Éxito con contenido de respuesta	GET, PUT, PATCH exitosos
201 Created	Recurso creado exitosamente	POST que crea un nuevo recurso; incluir header Location
202 Accepted	Solicitud aceptada para procesamiento asíncrono	POST /optimize cuando ALNS/IPSO se ejecuta en segundo plano
204 No Content	Éxito sin contenido de respuesta	DELETE exitoso
400 Bad Request	Error de validación de entrada	Payload con campos faltantes o inválidos
401 Unauthorized	Ausencia o invalidez de credenciales	Token JWT ausente o expirado
403 Forbidden	Autenticado pero sin permiso	Un conductor intenta acceder a rutas de otro conductor
404 Not Found	Recurso inexistente	GET /delivery-routes/{id} con id inexistente
409 Conflict	Conflicto de estado del recurso	Intentar asignar un vehículo ya asignado a otra ruta activa
422 Unprocessable Entity	Entrada válida sintácticamente pero inconsistente en negocio	Ruta sin factibilidad de capacidad de vehículo
500 Internal Server Error	Error no controlado del servidor	Excepción no prevista en el algoritmo de optimización

5. Estructura estándar de los JSON de respuesta

5.1. Respuesta exitosa

Toda respuesta exitosa envuelve el resultado en un objeto con la siguiente forma:

-

```
JSON
{
  "data": { ... },
  "meta": {
    "timestamp": "2026-09-15T10:30:00Z"
  }
}
```

- Para colecciones paginadas, meta incluye además page, pageSize, totalItems y nextCursor (paginación basada en cursor, según el estándar de base de datos).

5.2. Respuesta de error

Toda respuesta de error sigue una estructura homogénea, independiente del endpoint que la origine:

-

```
JSON
{
  "error": {
    "code": "ROUTE_INFEASIBLE",
    "message": "No se encontró una solución factible para la ruta solicitada",
    "details": [
      { "field": "vehicleCapacity", "issue": "excede la capacidad máxima" }
    ]
  }
}
```

- El campo code es un identificador estable en mayúsculas y guión bajo, consumible programáticamente por el frontend para mostrar mensajes específicos (ej. distinguir un error de validación de uno de infactibilidad del algoritmo).
- El campo message es legible por humanos y puede usarse directamente en la interfaz cuando no exista una traducción específica.
- El arreglo details es opcional y solo se incluye en errores de validación (400/422) con múltiples campos afectados.

5.3. Convenciones de nombres de campos

- Todos los campos JSON se escriben en camelCase, tanto en el request como en el response, para mantener consistencia directa con las variables de TypeScript del frontend.
- Las fechas y horas se serializan en formato ISO 8601 con zona horaria UTC (ej. 2026-09-15T10:30:00Z).
- Los identificadores de recursos se exponen como UUID en formato string, nunca como enteros autoincrementales, para no filtrar el volumen interno de datos.

6. Contratos y DTOs

- Cada endpoint expone DTOs (Request/Response) definidos explícitamente en el backend; nunca se serializan directamente las entidades JPA.
- Los contratos de API (esquema de cada DTO) deben documentarse con OpenAPI/Swagger, generado automáticamente a partir de las anotaciones del backend, y publicarse en un endpoint /api/v1/docs accesible para el equipo de frontend.
- Todo cambio incompatible en un contrato existente (renombrar o eliminar un campo, cambiar un tipo) requiere incrementar la versión del prefijo de la API o introducir el campo de forma aditiva y marcar el anterior como deprecated durante un periodo de transición.

7. Cabeceras y Seguridad

- Todas las peticiones requieren la cabecera Authorization: Bearer <token_jwt> para autenticar al usuario.
- Las solicitudes POST, PUT y PATCH deben incluir Content-Type: application/json.

8. Webhooks y Eventos Externos

- Los eventos asíncronos y notificaciones hacia sistemas externos se gestionan mediante suscripciones Webhook con firma HMAC para garantizar la integridad.

9. Referencias y recursos

Especificación OpenAPI: <https://swagger.io/specification/>

Guía de diseño de APIs REST de Microsoft: <https://github.com/microsoft/api-guidelines>

RFC 7807 — Problem Details for HTTP APIs: <https://www.rfc-editor.org/rfc/rfc7807>